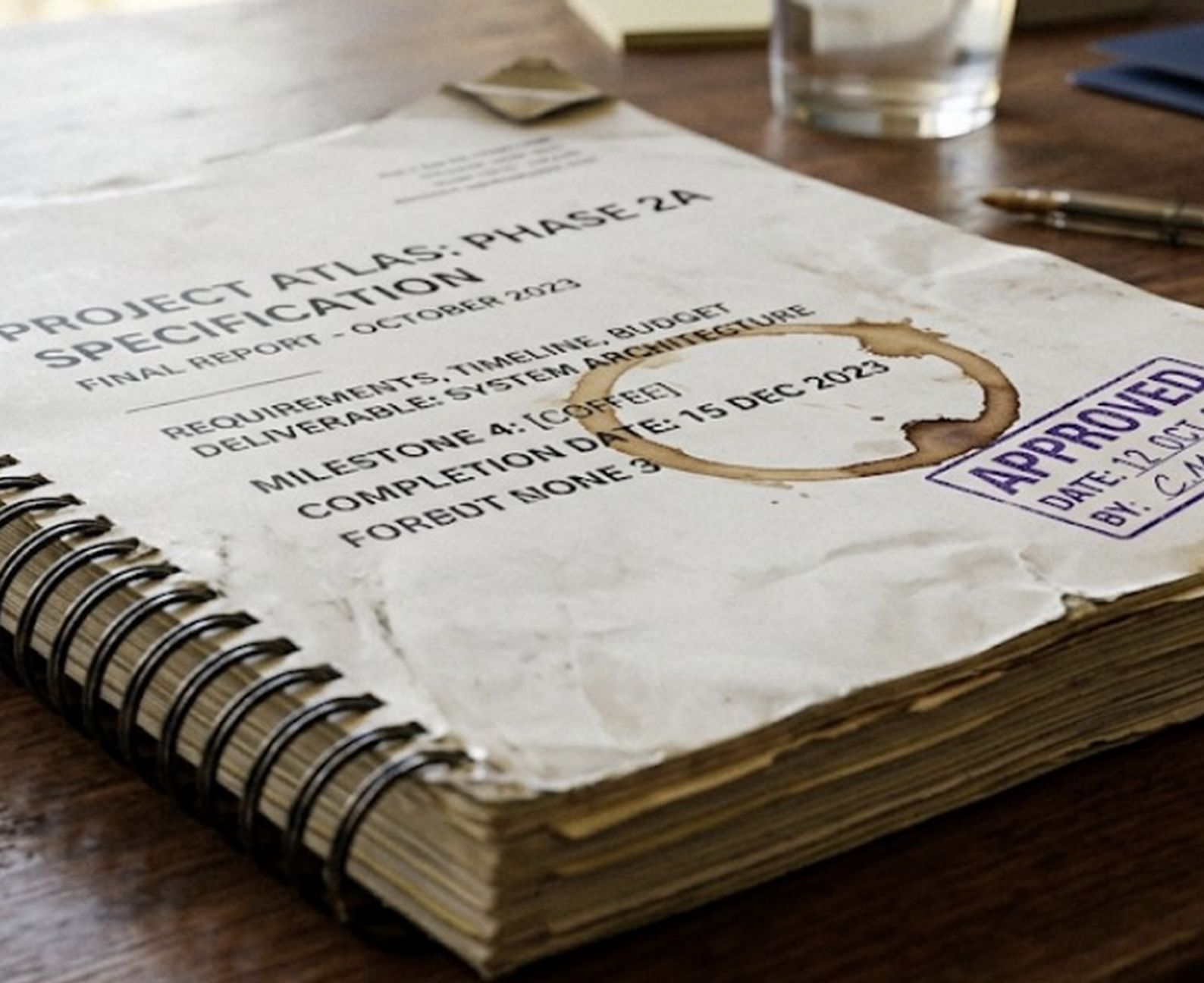




InvestPuppy
investpuppy.com

UNVARNISHED

Why financial software implementations fail

Serious when it matters.

About InvestPuppy

InvestPuppy builds Vektor, a systematic portfolio management platform for wealth management professionals.

What this series is

Every experienced practitioner thinks it. Almost nobody publishes it: *there has to be a better way.*

We thought it long enough that we went and built one.

Before that, we spent decades in the rooms.

The same project was failing for the third time. The project plan had been signed off before anyone had understood the first requirement.

The relationship was too warm for anyone to say so.

It isn't incompetence.

It's the incentive structure doing its job perfectly.

Nobody planned it this way. Nobody needed to.

We got frustrated enough to write it down. Then frustrated enough to build something. These papers are the writing-it-down part.

The something we built is a separate conversation.

The same failures repeat across every financial software implementation in this industry. The same conversations happen. The same silences happen. The same blame is distributed in the same directions. Nobody writes it down because nobody benefits from writing it down. This series does.

1. The Pattern

The pattern is recognisable to anyone who has been through one of these implementations. Not recognisable in retrospect — recognisable in real time, from week three, while it is still happening. The problem is not that the pattern is invisible. The problem is that by the time it is visible, the incentives that produced it are already in place.

A financial software implementation begins with optimism on both sides. The vendor has sold a capable platform. The client has bought a solution to a real problem. The project plan looks comprehensive. The timelines look achievable. The kick-off meeting produces the right kind of energy.

By week six, something is wrong. Not dramatically wrong — not a visible crisis — but wrong in the specific, quiet way that financial software implementations go wrong. A deadline has slipped. An integration point has turned out to be more complex than anticipated. A workflow assumption that seemed reasonable at signing has turned out not to reflect how the practice actually operates. The people who should be making decisions are not making them.

By month three, the project is in a different shape from the one that was sold and bought. The question of whose fault this is will eventually need an answer. The answer will be complicated.

This is not a story about a single implementation. It is the same story, told with different names on the door, across decades of financial services technology adoption.

2. The Failure Modes

Five patterns appear in every failed implementation. They rarely appear alone.

Nobody owns it.

At signing, the vendor believes the client owns the implementation. The client believes the vendor owns it. Both are correct. The accountability gap is not a misunderstanding — it is a structural feature of how these projects are sold and bought. The next paper in this series examines this in full.

The plan was built before anyone understood the workflow.

The implementation plan was written before week one. Before a single conversation about how the practice actually operates. Before the specific quirks of the client's approval process were understood. Before the edge cases in the data integration were mapped. The plan reflects what the vendor knows how to deliver. It does not reflect what the client

needs to receive.

Integration was the afterthought.

Every implementation has integration touchpoints — inbound data, outbound order files, connectivity to existing systems. In every failed implementation, these touchpoints were identified late, scoped inadequately, and tested under pressure at go-live. Integration is not a peripheral concern. It is the implementation. Treating it as a marginal consideration guarantees a marginal result.

Go-live was declared before confidence was earned.

The go-live date was in the contract. It was on the project plan. The pressure to hit it was real from both sides — the vendor needed to close the project, the client needed to demonstrate progress internally. The confidence required to actually go live — the practitioner's settled sense that the system is behaving correctly — was not present. The date was hit. The confidence was not.

The vendor relationship prevented honest conversation.

The sales relationship that brought the project into being is the same relationship that makes it difficult to say, six months in, that something is wrong. The warmth that was an asset in the sales process becomes a liability in the delivery process. The honest conversation that would have identified the problem in week three is the same conversation the relationship cannot afford.

3. Why They Keep Happening

These failures are not caused by incompetence. They are caused by incentive structures that make them the rational outcome.

The vendor's incentives are aligned toward signing, not toward implementation success. The sales team is measured on contracts, not on outcomes. The delivery team inherits the

commitments made in the sales process and is measured on hitting the timeline, not on whether the client's practice actually changed.

The client's incentives are aligned toward purchasing, not toward adoption. The person who bought the platform is measured on having bought a credible solution, not on whether their team uses it. The people who will use it are not always the people who decided to buy it.

The implementation fails in the space between these two incentive structures. Not because anyone planned for it to fail. Because neither side's incentives required it to succeed.

This matters because the fix is not found in the people. Better project managers, more experienced consultants, stronger governance — these mitigate the symptoms. They do not address the cause. The cause is structural. It will reproduce itself with every new project until the structure changes.

4. What This Series Is

This series is not a consulting framework. It is not a methodology. It is not a set of best practices derived from industry research. It is field notes — observations from inside the implementations, from both sides of the table, across enough projects to know that the patterns are structural, not accidental.

Each paper in this series takes one failure mode and examines it directly. Not to assign blame. Not to propose a universal fix. To name the mechanism accurately — so that the people sitting in those rooms can recognise it when it is happening, not in retrospect, but in real time.

This series locates the cause.